

Objektorientierung in PHP

10 Mini-Aufgaben · Startcode mit TODOs · Lösungen getrennt

Das Ziel

Klassen und Objekte unterscheiden, Attribute verwenden und Methoden aufrufen. Danach Konstruktor und Kapselung ergänzen. Erst die letzte Aufgabe verbindet OOP mit einem kleinen Formular.

Der Umfang bleibt klein

Keine Vererbung, keine Interfaces, kein Framework, keine Datenbank und kein MVC-Projekt. Aufgaben 1–9 arbeiten mit festen Werten. Für Aufgabe 10 sind HTML-Formular, POST-Prüfung und Fehler-Rückleitung bereits vorbereitet.

So startest du

Lege das gesamte Repository PHP_AI_DEMO unter C:\xampp\htdocs ab. Starte Apache und öffne in der Kursübersicht das optionale OOP-Material.

Einstieg im Repository: `Optionales_Zusatzmaterial/oop/index.html`

So arbeitest du

Öffne pro Aufgabe den eigenen Ordner in **aufgaben/**. Bearbeite nur die TODO-Stellen. Die Ausgabe nach HTML ist fertig; die Werte dafür sammelst du in `$ausgaben`. Prüfe nach jedem kleinen Schritt das Ergebnis.

Lernweg: Erst öffentlich zugängliche Attribute zum Verstehen der Syntax. Ab Aufgabe 7 werden die Attribute mit `private` gekapselt und über öffentliche Methoden verwendet.

Bezug zu deinen Unterlagen

Die PHP -BigPicture-Planung nennt OOP-Grundlagen vor MVC. Die Grundbegriffe knüpfen an die vorhandene Java-OO-Unterlage an. Die PHP-Beispiele und Aufgaben sind eine **neue Übertragung**, keine kopierten PHP-Folien. Eine ausgearbeitete PHP-OOP-Lektion ist in den verfügbaren PHP -Kapiteln 1–5 nicht enthalten. Herkunft und technische PHP-Referenzen: Seite 7 und QUELLEN.md.

01 Eine Lampe als Objekt

class · Attribut · new · ->

Klasse und erzeugtes Objekt unterscheiden.

Dateien: index.php

1. Ergänze in der Klasse Lampe das öffentliche Attribut farbe mit dem Startwert weiss.
2. Erzeuge außerhalb der Klasse genau ein Objekt in der Variablen \$lampe.
3. Lies die Farbe über das Objekt und übernimm sie in das vorbereitete Ausgabe-Array \$ausgaben.

Teste selbst

Die Seite zeigt weiss.

Ändere den Startwert in gelb. Nach dem Neuladen erscheint gelb.

02 Zwei Lampen, zwei Zustände

Zwei Instanzen derselben Klasse

Jedes getrennt erzeugte Objekt hat seinen eigenen Zustand.

Dateien: index.php

1. Erzeuge mit der vorgegebenen Klasse zwei getrennte Objekte: \$lampe1 und \$lampe2.
2. Setze die erste Farbe auf gelb und die zweite auf blau. Ändere danach nur die erste Farbe in gruen.
3. Übernimm die beiden Farben in dieser Reihenfolge in \$ausgaben. Für jedes Objekt verwendest du einen eigenen new-Aufruf.

Teste selbst

Die Ausgabe lautet zuerst gruen, darunter blau.

Eine Änderung an \$lampe1 verändert \$lampe2 nicht.

03 Ein Namensschild spricht

Methode · \$this · return

Eine Methode liest ein Attribut ihres eigenen Objekts.

Dateien: index.php

1. Die Klasse Namensschild besitzt bereits das Attribut name mit dem Wert Mia.
2. Ergänze getText(): Die Methode soll Hallo, Mia! zurückgeben und dabei den aktuellen Attributwert verwenden. Kein echo in der Methode.
3. Erzeuge ein Objekt und übernahm den Rückgabewert des Methodenaufrufs in \$ausgaben.

Teste selbst

Mit name = Mia erscheint Hallo, Mia!

Ändere den Attributwert auf Noah: Der Text muss sich ohne Änderung der Methode anpassen.

04 Ein Zähler zählt weiter

Parameter · Attribut verändern · Rückgabewert

Parameter und gespeicherten Attributwert auseinanderhalten.

Dateien: index.php

1. Der Zähler startet mit stand = 0. Ergänze erhoehen(\$schritt): Addiere den übergebenen Schritt zum aktuellen Stand und gib den neuen Stand zurück.
2. Erzeuge einen Zähler. Rufe die Methode zuerst mit 2 und danach mit 3 auf. Übernimm beide Rückgabewerte in \$ausgaben.
3. Verwende hier nur die vorgegebenen ganzen Zahlen. Eine eigene Gültigkeitsprüfung folgt in Aufgabe 8.

Teste selbst

Die beiden Ausgaben lauten 2 und 5, nicht 2 und 3.

Ein dritter Aufruf mit 0 lässt den Stand bei 5.

05 Ein Buch mit Startwert

__construct · Startwert beim Erzeugen

Einen Wert schon beim new-Aufruf mitgeben.

Dateien: index.php

1. Die Klasse Buch besitzt das Attribut titel. Ergänze den Konstruktor mit einem Parameter \$titel.
2. Übernimm den Parameter im Konstruktor in das Attribut des neuen Objekts.
3. Erzeuge \$buch mit dem Titel PHP-Einstieg. Übernimm anschließend den Titel in \$ausgaben; kein nachträgliches Setzen von außen.

Teste selbst

Direkt nach dem Erzeugen enthält das Buch PHP-Einstieg.

Übergib HTML-Basics beim Erzeugen: Die Ausgabe lautet HTML-Basics.

06 Ein Artikel mit zwei Angaben

Konstruktor mit zwei Parametern · Methode

Mehrere zusammengehörige Werte in einem Objekt bündeln.

Dateien: index.php

1. Ergänze Artikel mit den öffentlichen Attributen name und anzahl sowie einem Konstruktor für beide Startwerte.
2. Ergänze getInfo(): Die Methode gibt einen Text aus Name, Doppelpunkt und Anzahl zurück, beispielsweise Stift: 4.
3. Erzeuge zwei Artikel mit den vorgegebenen, gültigen Werten Stift / 4 und Heft / 2. Übernimm beide Methodenergebnisse in \$ausgaben.

Teste selbst

Es erscheinen Stift: 4 und Heft: 2.

Ändere nur die Anzahl des zweiten Artikels beim Erzeugen auf 0: Die erste Ausgabe bleibt unverändert.

07 Ein Gerät kapselt seinen Namen

private · public · Getter

Datenzugriff bewusst über eine Methode erlauben.

Dateien: index.php

1. Die Klasse Geraet soll ihren Namen Tablet nicht mehr öffentlich zugänglich machen. Ergänze das private Attribut name mit diesem Startwert.
2. Ergänze eine öffentliche Methode getName(), die den gespeicherten Namen zurückgibt.
3. Erzeuge ein Gerät und übernimm seinen Namen ausschließlich über die Methode in \$ausgaben.

Teste selbst

Die normale Ausgabe lautet Tablet.

Erkläre am Code: Welche Stelle darf von außen verwendet werden und welche nicht? Ein absichtlich eingebauter Direktzugriff gehört nicht in die Abgabe.

08 Lautstärke nur im erlaubten Bereich

Setter · Prüfung · true / false

Die Klasse schützt ihren gültigen Zustand; 0 ist erlaubt.

Dateien: index.php

1. Das private Attribut wert startet mit 5; getWert() ist vorbereitet. Ergänze setWert(\$wert).
2. Akzeptiere ausschließlich int-Werte von 0 bis 10. Bei Erfolg: Wert ändern und true zurückgeben. Sonst: false zurückgeben und den bisherigen Wert behalten.
3. Die Aufrufe für 0 und danach 11 sind vorgegeben. Ergänze die Methode und aktiviere den gekennzeichneten Testblock.

Teste selbst

setWert(0): angenommen; der gespeicherte Wert ist 0.

Danach setWert(11): abgelehnt; der Wert bleibt 0.

Auch -1, 2.5 und der Text "5" werden abgelehnt.

09 Das Buch bekommt eine eigene Datei

Klasse auslagern · require_once · verwenden

Bekannte Dateiaufteilung auf eine Klasse übertragen.

Dateien: Buch.php und index.php

1. Verschiebe die vorbereitete Klasse Buch aus dem markierten Bereich in die Datei Buch.php. Sie behält Konstruktor, privates Attribut und getTitle().
2. Binde die Klassendatei am Anfang von index.php über require_once und __DIR__ ein. In index.php steht danach keine eigene Definition der Klasse Buch mehr.
3. Erzeuge das Buch mit dem Titel Web-Grundlagen und übernahm den Rückgabewert von getTitle() in \$ausgaben.

Teste selbst

Die Ausgabe lautet Web-Grundlagen.

Prüfe die Aufteilung: Buch.php enthält die Klasse, index.php verwendet das Objekt und erzeugt die HTML-Ausgabe.

10 Vom kleinen Formular zum Objekt

POST · prüfen · Objekt erzeugen · Getter / Setter

OOP ergänzen, ohne den bekannten Formularablauf neu zu erfinden.

Dateien: index.php, auswertung.php, Lagerartikel.php und function.php

1. Das Formular mit Name und Bestand sowie POST-Prüfung und Fehler-Rückleitung sind vorbereitet. Lies den Ablauf: isset(), Texttyp, trim(), Pflichtangaben und ganze Bestandszahl von 0 bis 50.
2. Ergänze Lagerartikel: private Attribute name und bestand (Startwert 0), Konstruktor für den Namen, getName(), getBestand() und setBestand(\$bestand). Der Setter akzeptiert nur int von 0 bis 50, liefert true/false und behält bei Fehlern den bisherigen Wert.
3. Ergänze im markierten Erfolgsbereich der Auswertung: Objekt erst nach fehlerfreier Eingabeprüfung erzeugen, Bestand über den Setter übernehmen und beide Werte über Getter ausgeben. Kein Zugriff auf \$_POST in der Klasse.

Teste selbst

Stift / 0 → Erfolgsausgabe mit Bestand 0.

Stift / 51 oder abc → 303-Rückleitung, Name bleibt, Bestand leer.

Name leer / 4 → Bestand bleibt; beide leer → beide Meldungen. Direkter GET-Aufruf der Auswertung darf keinen Erfolg auslösen.

Merkhilfe für die Übungen

Nur die Bausteine – keine vollständigen Aufgabenlösungen.

PHP-Baustein	Bedeutung
<code>class Name { ... }</code>	Eine Klasse definieren.
<code>public \$wert = 0;</code>	Ein öffentliches Attribut mit Startwert.
<code>\$objekt = new Name();</code>	Ein konkretes Objekt erzeugen.
<code>\$objekt->wert</code>	Ein öffentliches Attribut dieses Objekts lesen.
<code>\$objekt->methode()</code>	Eine Methode dieses Objekts aufrufen.
<code>\$this->wert</code>	Das eigene Attribut innerhalb einer Objektmethode.
<code>public function __construct(\$wert)</code>	PHP-Konstruktor mit einem Startparameter.
<code>private \$wert = 0;</code>	Attribut innerhalb seiner Klasse kapseln.
<code>return \$wert;</code>	Einen Wert an den Aufrufer zurückgeben.

Die wichtigsten Unterschiede

Klasse ≠ Objekt: Die Klasse ist der Bauplan. Zwei getrennte new-Aufrufe erzeugen zwei Objekte. Eine Zuweisung an eine zweite Variable erzeugt keine unabhängige Kopie. [1]

Parameter ≠ Attribut: In `$this->wert = $wert;` ist links das Attribut und rechts der Parameter. Bei einem festen Attributnamen steht nach dem Pfeil kein zweites Dollarzeichen. [1, 2]

Getter / Setter: Namen für Lese- und Änderungsmethoden. Die true-/false-Rückgabe des Setters ist hier eine Übungsregel. Ungültige Werte dürfen den bisherigen Zustand nicht verändern. [4]

Konstruktor: Heißt in PHP `__construct` und läuft beim Erzeugen. Die Java-Schreibweise aus der Vorlage wird nicht übernommen. [3]

Quellen und Abgrenzung

Deine Unterlagen: BigPicture_PHP .pdf, S. 4–5; Java-OO, PDF-S. 9–11, 15–16 und 43 (gedruckte Folie 44), außerdem Folie „Attribute“ (gedruckte Folie 62). Qualitätssicherung, S. 8 und 22: klare Soll-Ergebnisse und modularer Code. Der Formularablauf knüpft an das Alltagsformulare-Paket an.

Zusätzliche PHP-Syntaxreferenzen: [1] Grundlagen, [2] Eigenschaften, [3] Konstruktoren, [4] Sichtbarkeit. Alle aus dem offiziellen PHP-Handbuch. Die Webvariante und QUELLEN.md enthalten die anklickbaren vollständigen Links. Beispiele und Aufgaben sind neu erstellt.